INVESTIGACIÓN EXPERIMENTAL · HPC

Cholesky

De la organización de memoria a la evaluación de MPI frente a LAPACK

Implementación en C · distribución MPI · referencia optimizada

Facultad de Ciencias Exactas y Tecnología · Universidad Nacional de Tucumán

Pregunta central

Cuando distribuir el trabajo reduce el tiempo y cuánto depende esa ventaja del kernel local y de la frontera de medición.

Índice de lectura

1. Resultados principales 3

2. Resumen 4

3. Problema matemático y primera implementación verificable 4

4. La primera decisión cambió al estudiar los accesos a memoria 5

5. Distribuir columnas y conservar trabajo útil 7

6. Ampliar la frontera: entrada, resultado y generación 9

7. La granularidad de propiedad no produjo una mejora estable 9

8. LAPACK local mostró que faltaba eficiencia dentro del kernel 10

9. Consolidación del candidato y repetición local 11

10. Definir el reloj antes de interpretar el speedup 12

11. Transición al clúster: recursos visibles y recursos asignados 13

12. Cruce formal del tiempo de aplicación 14

13. Escalabilidad y beneficio del comando completo 16

14. La referencia optimizada en el clúster 18

15. Comparación final de problemas grandes 19

16. Extensión de strong scaling hasta 32 procesos 22

17. Corrección numérica e integridad de la evidencia 25

18. Condiciones del entorno y límites de atribución 26

19. Evaluacion en una segunda computadora 28

20. Conclusion 29

21. Continuación técnica razonable 30

1. Resultados principales

La pregunta experimental es cuándo MPI reduce el tiempo y cuánto depende esa ventaja del cálculo local. Los tres resultados siguientes pertenecen a campañas y relojes identificados; no forman una única curva combinada.

Resultado	Evidencia e interpretación
Primer cruce	N=192, P=3: primer tamaño ensayado con mejora descriptiva clara de application_s
El reloj cambia la	Con P=6, la campaña de escalabilidad mejora el comando completo en N=2048 y N=4096; en N=512 y N=1024 la mejora de aplicación no alcanza para ganar en external_s.
LAPACK conserva la	En factor_s, MPI P=6 tarda 8,58 veces LAPACK de un hilo para N=6144 y 6,25 veces
Strong scaling	Para N=8192, factor_s baja de 64,11 s con MPI P=1 a 13,54 s con P=32: speedup

Candidato seleccionado: right-looking en double, vector local, columnas 1D cíclicas b=1, generación distribuida y reunión del factor en rango 0.

Condiciones de lectura: experimentos locales y campañas del clúster separados. Las primeras sesiones utilizaron una cuota agregada de 6 CPUs y 24 GiB; la sesión siguiente dispuso de 48 CPUs equivalentes y 192 GiB. Las barras Q1-Q3 describen dispersión de las repeticiones, no intervalos de confianza.

Recorrido del informe: primero se explica el algoritmo y la selección del candidato; después se distinguen relojes y resultados del clúster; el cierre reúne corrección, límites y conclusiones.

2. Resumen

Se estudió cómo el tamaño de una matriz simétrica definida positiva modifica la conveniencia de una implementación secuencial propia, una implementación MPI y una referencia local LAPACK/OpenBLAS de un hilo (sin MPI). El trabajo comenzó con la corrección de Cholesky y evolucionó mediante comparaciones controladas de algoritmos, accesos a memoria, propiedad de columnas, generación y fronteras de medición. Las pruebas locales permitieron seleccionar un candidato right-looking en C, con vector auxiliar, distribución 1D cíclica por columnas y generación distribuida. Las campañas evaluaron cruce, escalabilidad hasta P=6 y distancia frente a LAPACK; una sesión independiente extendió el strong scaling de N=8192 hasta P=32.

Los resultados muestran una mejora de MPI respecto del secuencial propio para parte de la grilla, pero no una ganancia automática al aumentar los procesos. El primer tamaño ensayado con mejora descriptiva clara de aplicación fue N=192 con P=3. En la campaña de escalabilidad, P=6 aceleró la aplicación en los cuatro tamaños estudiados, mientras que el comando completo solo mejoró en N=2048 y N=4096. La comparación final aisló la fase de factorización: en N=6144 y N=8192, MPI P=6 redujo el tiempo del kernel propio, pero continuó siendo más lento que LAPACK/OpenBLAS de un hilo. No se observó un cruce entre estas dos implementaciones hasta N=8192. En la extensión, P=32 fue la configuración más rápida ensayada para N=8192, aunque con escalabilidad sublineal y una degradación reproducida entre P=4 y P=8.

La contribución del proyecto consiste en delimitar esas ganancias y sus condiciones, conservar los resultados negativos y mostrar por qué mejorar el cálculo local y distribuirlo son decisiones complementarias. Las conclusiones corresponden a matrices generadas, versiones concretas y sesiones identificadas. No constituyen una evaluación multinodo ni una comparación con ScaLAPACK.

3. Problema matemático y primera implementación verificable

Para una matriz real simétrica definida positiva A, se busca un factor triangular inferior L con diagonal positiva tal que $A = L\,L^T$. El proyecto calcula el factor; no implementa la solución completa de un sistema Ax=b. La dependencia entre columnas se obtiene de la identidad del producto: el pivote de la columna j depende de contribuciones anteriores y los elementos inferiores utilizan ese pivote.

En la formulación left-looking se calcula l(j,j) como la raíz de a(j,j) menos la suma de cuadrados de la fila ya calculada. Después, cada l(i,j) resta el producto escalar de las partes conocidas de las filas i y j y divide por l(j,j). Right-looking aplica esas contribuciones por adelantado a la región restante: termina una columna y actualiza los elementos todavía no factorizados. Ambas variantes mantienen un trabajo aritmético dominante de orden N^3, aproximadamente $N^3/3$ operaciones; su orden de acceso y sus necesidades de comunicación pueden diferir.

El generador C produce entradas simétricas fuera de la diagonal en [-1,1) y establece a(i,i)=1+sumatoria de valores absolutos fuera de la diagonal de la fila. La simetría y la dominancia diagonal estricta positiva justifican la definición positiva en aritmética exacta. Esta familia facilita repetir el experimento y comprobar la implementación. No cubre matrices arbitrariamente mal condicionadas, matrices dispersas ni entradas procedentes de aplicaciones reales.

La representación es densa, en double, almacenada por filas. La factorización sobrescribe el triángulo inferior; el superior conserva datos que no pertenecen al factor L. La reserva sigue siendo de N^2 elementos, aunque solo el triángulo inferior sea el resultado. Las rutinas rechazan pivotes no positivos o no finitos; su contrato presupone una entrada simétrica. No debe interpretarse ese control de pivotes como una comprobación completa de simetría para entradas externas.

Las pruebas utilizan el ejemplo A = [4,12,-16; 12,37,-43; -16,-43,98], cuyo factor inferior es [2,0,0; 6,1,0; -8,5,3]. También comprueban escalares, identidad, tamaños no redondos, distintas semillas, entradas indefinidas o semidefinidas y valores no finitos. Un factor alterado deliberadamente permite verificar que la comprobación detecta errores.

$$A = LL^{\mathsf{T}}$$

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}} \quad (i > j)$$

4. La primera decisión cambió al estudiar los accesos a memoria

La comparación inicial de left-looking y right-looking en C no respaldó elegir right-looking por velocidad secuencial. Con siete repeticiones por variante, una preparación inicial y orden alternado, left-looking fue más rápido en casi todos los tamaños ensayados. En N=1024 las medianas fueron aproximadamente 137,69 ms para left y 1109,62 ms para right. El reloj incluía solamente la factorización; generación, copia de entrada y reconstrucción quedaban fuera.

La inspección mostró una diferencia concreta: left recorría filas contiguas para sus productos escalares, mientras que el bucle interno de right releía a[k*n+j], una columna con saltos de N elementos. Esto motivó copiar la columna activa a un vector contiguo. El cambio no alteró los propietarios, las dependencias ni las actualizaciones matemáticas; añadió una copia de orden N^2 en una operación cuyo trabajo principal es de orden N^3. La reserva del vector quedó fuera del reloj y su llenado dentro.



Figura 1. Acceso por columna y copia contigua del kernel secuencial. Esquema conceptual; no representa contadores ni tiempos medidos

La nueva campaña volvió a medir las tres variantes juntas, sin reutilizar la referencia anterior: seis tamaños, incluida N=1000, doce repeticiones por variante y las seis permutaciones del orden repetidas dos veces.

N	Left (ms)	Right (ms)	Vector (ms)
64	0.024	0.023	0.020
128	0.200	0.257	0.172
256	1.838	5.816	1.391
512	18.022	185.696	12.325
1000	131.674	95.735	59.854
1024	144.791	1159.089	76.918

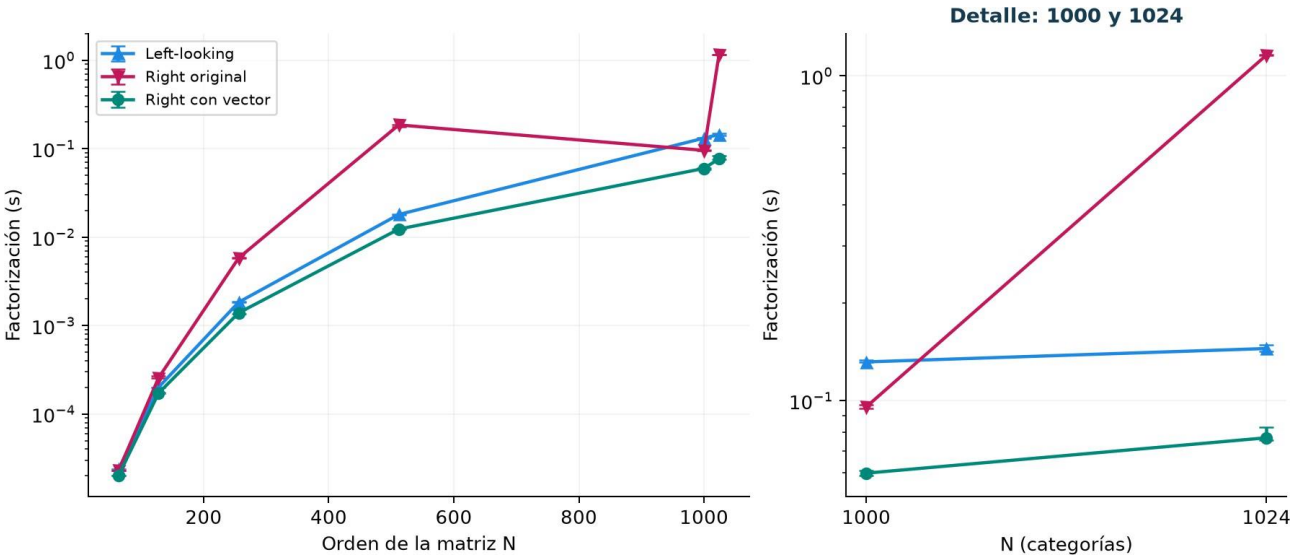


Figura 2. Medianas y Q1-Q3 de la campaña local de memoria, en segundos y escala logarítmica. Cada comparación pertenece a esa misma sesión.

El vector redujo la mediana en todos los tamaños de esa campaña. En N=1024, right original pasó de unos 1159,09 ms a 76,92 ms y también mejoró frente a left, que tardó 144,79 ms en esta sesión. Se eligió right con vector como referencia para desarrollar MPI. La conclusión se refiere a estas implementaciones; no establece la superioridad general de una formulación de Cholesky.

El salto de right original entre N=1000 y N=1024 también aconsejó evitar una interpretación basada solo en el número de operaciones. La organización de accesos era relevante, pero no se midieron contadores que permitieran atribuir la mejora exclusivamente a fallos de caché. La copia también puede modificar la vectorización y las decisiones del compilador.

5. Distribuir columnas y conservar trabajo útil

La primera implementación MPI asignó un propietario fijo a cada columna. El propietario calcula el pivote y el segmento inferior de la columna activa; MPI_Bcast distribuye ese segmento y cada proceso actualiza las columnas futuras que posee. Hay N difusiones durante una factorización completa, con mensajes decrecientes. Las comunicaciones y sus esperas pertenecen al kernel; no se interpretan como latencia pura de red.

Se compararon asignaciones contiguas y cíclicas. En la primera, cada proceso recibe un intervalo de columnas. En la segunda, la columna global j pertenece a j módulo P. Los datos locales se almacenan por filas de columnas propias: local[i*cols+c]. Por tanto, distribución por columnas no significa almacenamiento local column-major.

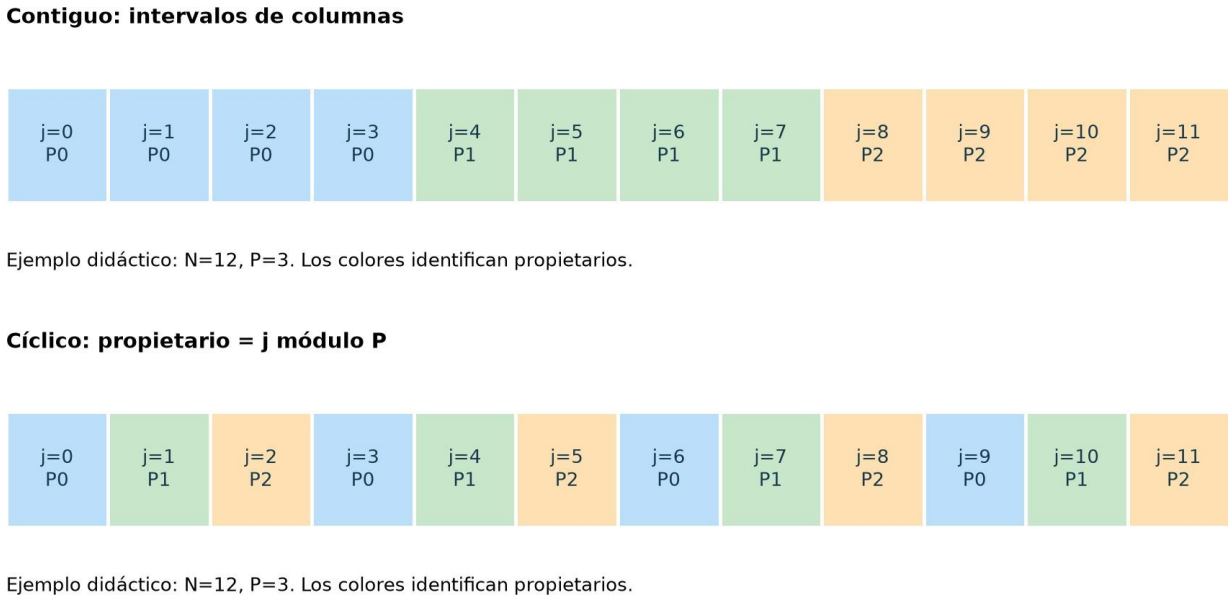


Figura 3. Propiedad contigua y cíclica para N=12 y P=3. En el candidato, el propietario de la columna activa calcula su segmento inferior y lo difunde a todos los procesos mediante MPI_Bcast; cada proceso actualiza sus columnas futuras.

El conteo acumulado de actualizaciones de una columna k es k(N-k). Igual número de columnas no implica igual trabajo ni igual actividad en cada momento. El reparto cíclico mantiene columnas futuras en varios procesos durante más tiempo, pero también introduce indexación y accesos que pueden resultar costosos. El conteo es una explicación estructural; no predice por sí solo el tiempo.

La primera campaña confirmó esa cautela. En N=1024/P=3, el contiguo tuvo una mediana de 40,03 ms y el cíclico de 51,67 ms. La hipótesis favorable al reparto cíclico no bastó para que la primera implementación ganara. Se conservaron ambos resultados y se examinó el acceso interno al vector difundido.

5.1. El vector local cambió el resultado del reparto

La campaña comparó cuatro variantes: contigua y cíclica, cada una con kernel original o vector local. Ocho repeticiones y un orden balanceado de cuatro variantes permitieron mantener la comparación contemporánea. Para N=1024/P=3:

Reparto y kernel	Mediana (ms)	Q1 (ms)	Q3 (ms)
contiguous:local	39.660	39.128	46.085
contiguous:original	39.615	39.060	51.169
cyclic:local	32.765	30.416	42.178
cyclic:original	72.041	69.947	79.077

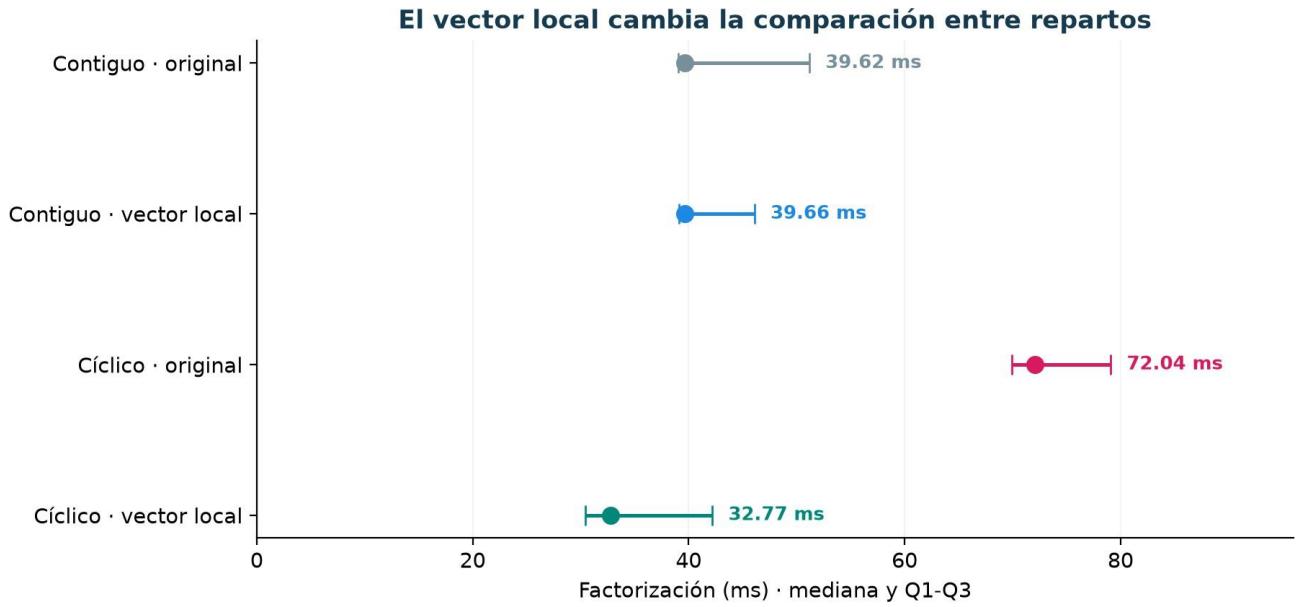


Figura 4. N=1024, P=3, campaña mpi_local_20260921; ocho repeticiones por variante. Punto: mediana; intervalo: Q1-Q3. Datos de data/processed/summary_all.csv, métrica seconds, correspondientes a la campaña local.

El cíclico con vector local alcanzó aproximadamente 32,77 ms frente a 72,04 ms del cíclico original medido en esa misma campaña. El contiguo con vector quedó alrededor de 39,66 ms. Así, la elección final del reparto surgió de combinar propiedad de datos y acceso local; no de asumir que una distribución equilibrada debía ser automáticamente más rápida. La referencia secuencial se midió antes y después dentro de la misma campaña.

Las pruebas MPI incluyeron N menor que P, columnas desiguales, factor conocido y fallos colectivos por pivote inválido. Los perfiles instrumentados se ejecutaron por separado y no reemplazan las mediciones principales sin instrumentación interna.

5.2. El primer cruce local y la comparación de algoritmos MPI

El estudio local del cruce utilizó N=64,96,128,160,192,224,256 y P=1,2,3. Se hicieron tres rondas con distinto orden de tamaños, ocho repeticiones MPI por variante y referencias secuenciales antes y después. Se exigió speedup mayor que 1,10 y Q3 de MPI menor que Q1 del mejor secuencial contemporáneo en las tres rondas. N=256/P=3 cumplió ese criterio; no se generalizó el resultado a otro reloj o máquina. Las tres rondas eran lanzamientos distintos dentro de una sesión, no tres días independientes.

Después se implementó left-looking MPI sobre el mismo reparto cíclico para contrastar la decisión algorítmica. Cada proceso calcula contribuciones con sus columnas anteriores y las reduce al propietario de la columna activa. El código realiza N reducciones escalares, N-1 reducciones vectoriales y N difusiones del pivote: 3N-1 colectivas, frente a N difusiones del right implementado. El conteo corresponde a estos códigos, no a toda implementación posible de left-looking.

En N=1024/P=3, las medianas fueron 54,08 ms para left MPI y 31,48 ms para right con vector local. La campaña alternó los algoritmos y mantuvo distribución y recolección fuera del reloj del kernel. El resultado sostuvo la selección de right dentro del diseño estudiado; no aisló cuánto de la diferencia se debía a colectivas, aritmética o accesos.

6. Ampliar la frontera: entrada, resultado y generación

Medir solo el kernel podía favorecer una solución que requiriera mucho trabajo adicional para preparar y devolver los datos. El siguiente banco definió un caso completo: A comienza en rango 0 y L debe terminar allí. Para MPI se midieron empaquetado y MPI_Scatterv, factorización, MPI_Gatherv y reconstrucción. Para el secuencial se midieron copia y factorización. Reserva, generación y validación todavía estaban fuera.

En N=1024/P=3, right MPI registró medianas de aproximadamente 5,323 ms de entrada, 38,848 ms de factor y 5,630 ms de salida; el total medido directamente fue 49,699 ms frente a 94,523 ms del secuencial de ese banco. No se obtiene ese total sumando medianas: cada fase se redujo por máximo entre procesos y esos máximos pueden corresponder a rangos distintos. En esta grilla el primer tamaño con una mejora total clara pasó a N=512, mostrando que un cruce del kernel no es un cruce del flujo completo.

6.1. Generación distribuida sin cambiar la matriz

La generación centralizada añadía un paso serial y el traslado inicial de la matriz. Se introdujo un generador indexado que reproduce el valor correspondiente a cada par de índices usando la misma secuencia entera y semilla 2026. Cada proceso genera sus columnas y calcula la diagonal sumando los valores absolutos en el orden de índices previsto. El banco contrasta el almacenamiento local contra columnas de la referencia centralizada mediante igualdad bit a bit.

Para N=2048/P=6, generar y distribuir pasó de una mediana de 51,862 ms a 8,385 ms en ese experimento. El resultado no significaba una mejora igual del programa completo: la generación es de orden N^2 y la factorización continúa dominando al aumentar N. Con un solo proceso, la generación distribuida incluso fue más lenta en ese tamaño, lo que se conserva como resultado negativo.

Se amplió entonces el reloj a reserva, generación, eventual distribución, kernel y reunión. En N=2048/P=6, el total centralizado fue 874,604 ms y el distribuido 837,392 ms; la referencia secuencial del mismo banco fue 1617,307 ms. La ganancia del flujo fue menor que la observada al aislar generación. Se adoptó la generación distribuida porque elimina la necesidad de construir y repartir A inicialmente desde rango 0, conservando una comparación numérica verificable. El resultado sigue reuniéndose al final y la validación regenera A en rango 0.

7. La granularidad de propiedad no produjo una mejora estable

Con el flujo completo disponible, se probó asignar grupos de b columnas consecutivas por turnos, con b=1,2,4,8,16,32,64. Cambió el mapa propietario/global/local, pero siguió existiendo una factorización escalar y una difusión por columna. Este experimento de propiedad no implementó paneles, productos matriz-matriz ni un Cholesky bloqueado basado en BLAS-3.

La campaña inicial incluyó N=512,1024,2048, P=1,3,6 y siete repeticiones por variante, con 441 observaciones. b=4 apareció favorable para N=1024/P=6 y se repitió conservando b=1 como

referencia. En nuevos lanzamientos de la misma sesión, las medianas fueron 48,93 y 44,02 ms, respectivamente. En N=2048 la diferencia quedó alrededor de 0,2 %, insuficiente para sostener una ventaja práctica estable.

La prueba de confirmación evaluó únicamente b=1 y b=4, con ocho repeticiones alternadas. El resultado cambió:

N	b=1 total (ms)	b=4 total (ms)
1024	42.339	43.359
2048	767.989	798.865

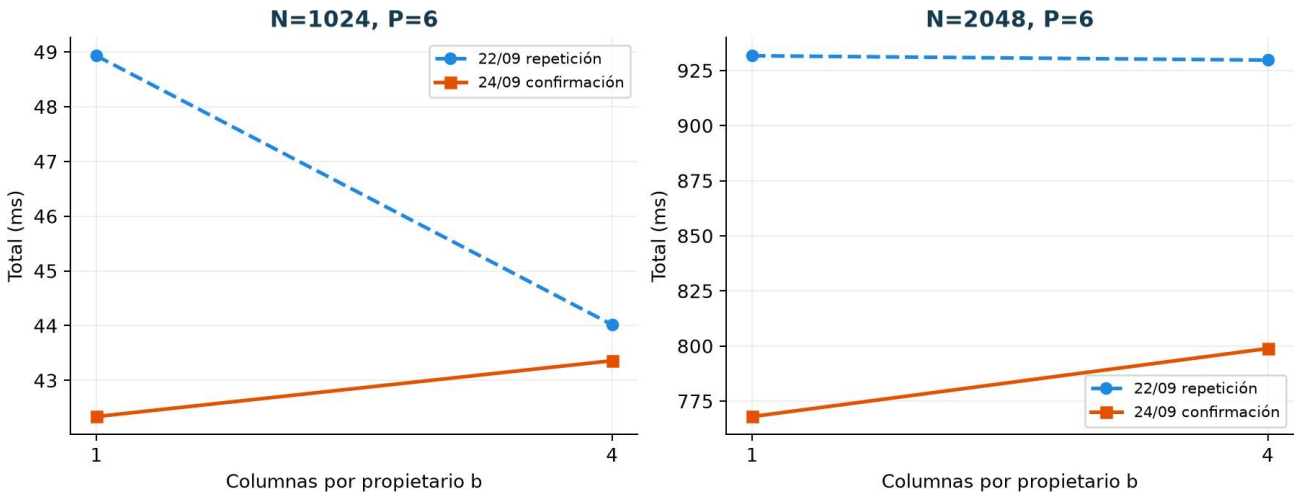


Figura 5. Comparaciones de b dentro de cada sesión, P=6. Las líneas separan la repetición del 22/09 y la confirmación del 24/09; no representan una única muestra combinada.

La ventaja inicial no se sostuvo entre sesiones. Se mantuvo b=1 como configuración principal y se conservó b=4 como alternativa experimental, sin buscar otro mínimo retrospectivo para reemplazarla. N=3072 se probó con dos repeticiones por variante como piloto de factibilidad; no fue una campaña suficiente para seleccionar un reparto.

8. LAPACK local mostró que faltaba eficiencia dentro del kernel

La comparación local incorporó una referencia profesional sin reemplazar la implementación educativa. LAPACK ofrece rutinas de álgebra lineal; DPOTRF calcula Cholesky. Su implementación de referencia utiliza bloques y llamadas BLAS de nivel 3, como operaciones sobre submatrices. LAPACKE proporciona la interfaz C. OpenBLAS aporta implementaciones optimizadas de esas operaciones. La combinación permite reutilización de datos y kernels especializados, pero el experimento no separa la contribución de bloqueo, vectorización, instrucciones, compilación y demás decisiones internas. DPOTRF, Netlib.

compare_lapack.c compara el right con vector propio con LAPACKE_dpotrf_work. Ambos usan double, el mismo generador y la misma semilla. Se fija OpenBLAS a un hilo por API y se verifica con openblas_get_num_threads. En la campaña local ambos métodos compartieron afinidad CPU 0; las versiones registradas fueron LAPACKE 3.12.1 y OpenBLAS 0.3.32.

La matriz simétrica row-major se interpreta como column-major con triángulo superior. El factor U que escribe LAPACK representa $L=U^T$ al leer el arreglo por filas. Esa correspondencia evita una transposición adicional y se comprueba con el factor conocido. La interfaz _work evita la conversión row-major y el

barrido opcional de NaN de la interfaz superior; no vuelve idénticos los contratos de error de ambas rutinas.

Se realizaron seis observaciones útiles por método para N=512,1024,2048, alternando el orden tras una preparación interna. N=3072 tuvo dos y se conserva como piloto. En N=2048, factor_s propio fue aproximadamente 1484,35 ms y LAPACK 116,30 ms: una relación de 12,76. El cociente del total del banco fue 9,74, porque reserva y generación permanecen en el flujo.

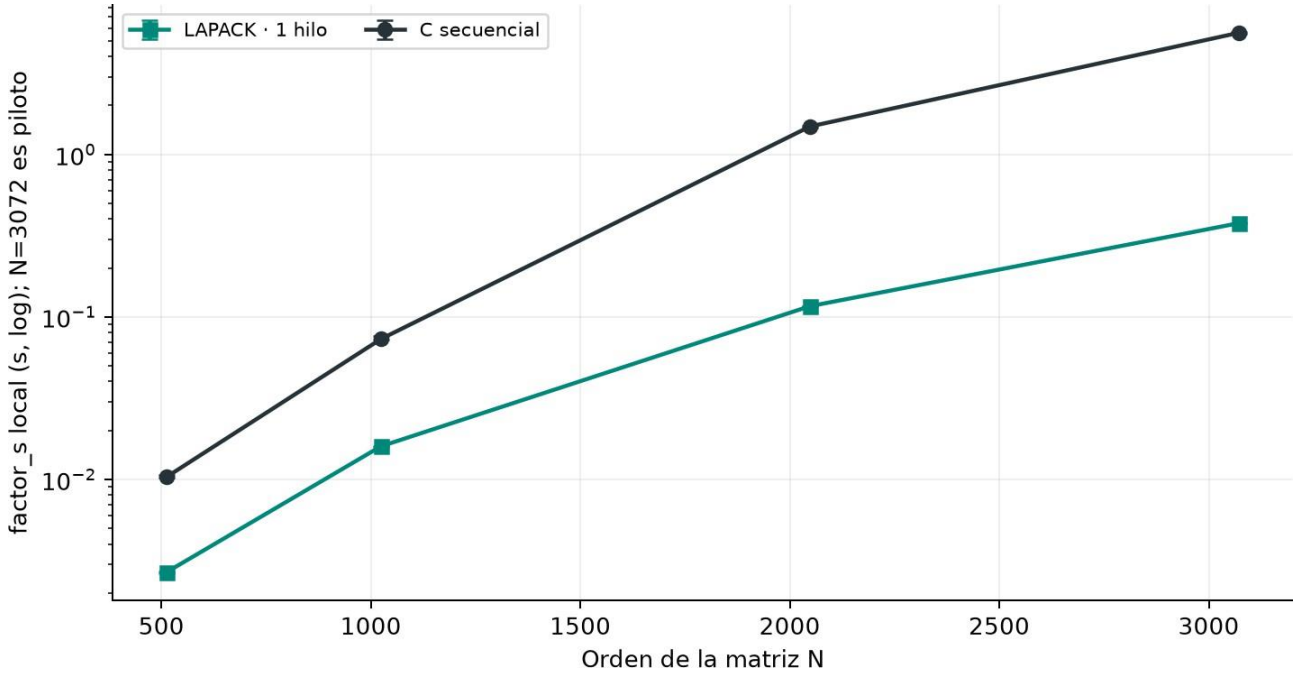


Figura 6. Comparación local C/LAPACK de factorización. N=3072 tiene solo dos observaciones por método y es exploratorio; no se usa para una conclusión formal de escalabilidad.

Estos resultados justifican estudiar optimizaciones del cálculo local, particularmente bloqueo y BLAS-3. La comparación no invalida el enfoque MPI: la eficiencia del kernel y la distribución entre procesos son aspectos complementarios que podrían combinarse en una implementación futura.

9. Consolidación del candidato y repetición local

main_candidate.c reunió las decisiones elegidas en un programa con dos compilaciones. Sin USE_MPI, genera A y ejecuta cholesky_right_buffered; con USE_MPI, genera columnas cíclicas, llama a cholesky_mpi_local y reúne el factor en rango 0. La propiedad es b=1. El candidato no usa OpenMP, no llama a LAPACK y no elige P automáticamente.

Recibe N entre 1 y 20000, utiliza semilla 2026 y devuelve métricas CSV. El límite de interfaz no garantiza que el problema quepa en memoria. La matriz L queda en memoria del rango 0; el programa no lee ni escribe matrices completas a disco. Además del almacenamiento distribuido mantiene buffers completos en rango 0 para reunión y validación, por lo que no representa una implementación de memoria mínima.

Las campañas locales utilizaron N=256,512,1024,2048 y los métodos secuenciales, MPI P=1,2,3,4,6. Cada sesión conservó 192 lanzamientos: 24 preparaciones y 168 observaciones útiles. Hubo siete repeticiones por configuración, rotación de métodos y 30 configuraciones pequeñas de comprobación adicionales. Los tamaños siguieron orden fijo, no una aleatorización completa.

Sesión	Reloj	MPI P6 (s)	Speedup	E (%)
20260924	application_s	0.890	1.825	30.4
20260924	external_s	1.685	1.161	19.4
20260927	application_s	0.906	1.953	32.6
20260927	external_s	1.807	1.159	19.3

La repetición sostuvo la dirección de la mejora para N=2048/P=6, aunque cambiaron los tiempos y el speedup interno. El incremento de recursos no produjo curvas siempre monótonas. Ambas sesiones se conservan separadas; reunirlas como si fueran catorce repeticiones homogéneas ocultaría esa variación.

10. Definir el reloj antes de interpretar el speedup

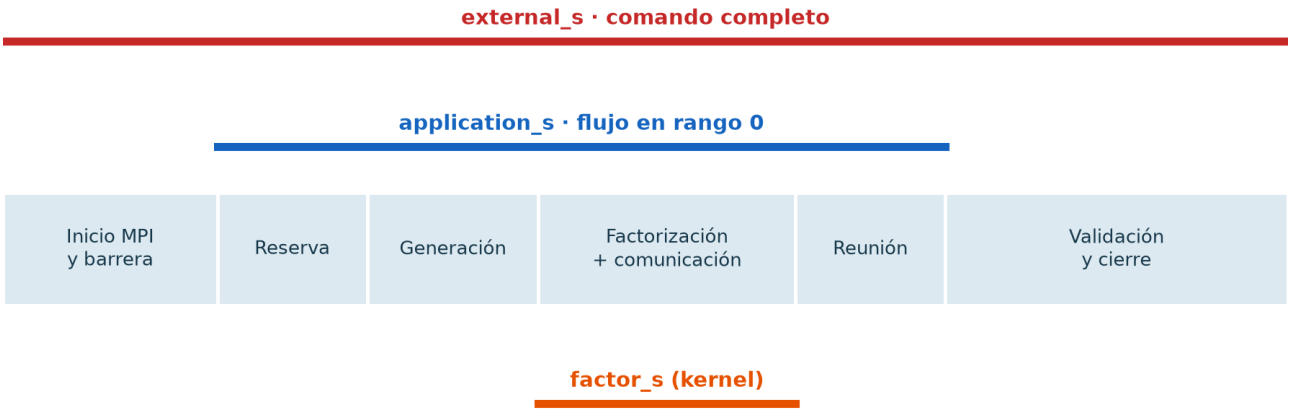


Figura 7. Relojes del candidato MPI, de la fase matemática al comando completo. Los anchos son ilustrativos. La inicialización queda fuera de application_s; la comunicación del kernel queda dentro de factor_s.

El candidato inicia MPI, establece el comunicador y realiza una barrera antes de comenzar el reloj de aplicación. En rango 0, allocation_s, generation_s, factor_s y gather_s son intervalos consecutivos. Su suma por observación coincide con application_s. En MPI, gather_s incluye tanto MPI_Gatherv como la reconstrucción serial del arreglo global en rango 0.

factor_s de MPI es la duración local observada en rango 0 de la llamada al kernel, con comunicaciones y esperas. No es cómputo puro ni un máximo reducido entre todos los rangos. max_local_s registra el máximo de las duraciones locales de todo el flujo, después del reloj principal; no redefine cada una de sus fases. La reducción de tiempos y la combinación del estado de fallo quedan fuera de application_s.

La validación comienza después. Regenera A en rango 0, calcula residuos y comprueba finitud y diagonal positiva; su duración es validation_s. Después se imprime el CSV, se comunica el estado final, se libera memoria y se finaliza MPI. Ni MPI_Init ni MPI_Finalize pertenecen a application_s.

measure_command.c mide con reloj monotónico desde antes de fork hasta el retorno de waitpid. El hijo ejecuta el comando completo mediante execvp; para MPI, ese comando incluye mpirun. Su CSV contiene external_s y el código de salida. El reloj incluye inicialización, programa, validación, salida y

cierre; excluye escribir el CSV del cronómetro. Tampoco incluye arrancar WSL, obtener una sesión web o esperar en una cola.

Por ello, external_s - application_s no equivale a startup MPI. Si además se resta la validación por observación, todavía quedan inicialización, barreras, reducciones auxiliares, liberación, salida y finalización. No se restan medianas independientes para atribuir esos componentes.

En compare_lapack.c, factor_s comprende la llamada local de factorización y total_s incluye reserva, generación y factorización, sin las referencias preparadas por el banco, la validación ni la liberación. El factor_s de MPI incluye las comunicaciones y esperas de esa fase, registradas en el rango 0. Se comparan, por tanto, tiempos de factorización de implementaciones distintas, no costos aritméticos aislados; total_s y application_s no son directamente equivalentes.

Se informan dos cocientes diferentes. La aceleración frente al secuencial propio es S_app(N,P)=mediana(T_seq,propio(N))/mediana(T_MPI(N,P)); su eficiencia descriptiva es E_app=S_app/P. La escalabilidad interna de MPI se expresa como S_MPI(N,P)=mediana(T_MPI(N,1))/mediana(T_MPI(N,P)). Como MPI con P=1 y el programa secuencial no siguen exactamente la misma ruta, estos cocientes no son intercambiables. En ninguno de los casos E expresa utilización real de CPU. Con N fijo, variar P corresponde a escalabilidad fuerte.

$$S = \frac{\text{mediana}(T_{\text{seq}})}{\text{mediana}(T_{\text{MPI}})} \qquad E = \frac{S}{P}$$

11. Transición al clúster: recursos visibles y recursos asignados

El clúster mostró un AMD EPYC 7B12, dos sockets de 64 núcleos, 128 CPUs visibles y dos dominios NUMA, con cpuset permitido 0-127. Las campañas principales se realizaron bajo cpu.max = 600000 100000, cuota agregada equivalente a seis CPUs por período, y memory.max = 25769803776, equivalente a 24 GiB. Antes se había utilizado una asignación de dos CPUs y 6 GiB durante la preparación.

Ver 128 CPUs o aproximadamente 503 GiB de memoria del host no concedía esos recursos a la sesión. El cpuset indica dónde puede ejecutarse un proceso; la cuota limita el tiempo agregado de CPU del cgroup. No implica seis núcleos exclusivos. La afinidad solicita ubicación y tampoco elimina interferencia del host. El diagnóstico archivado de P=6 muestra ranks en el primer socket, pero no sustituye registros por ejecución ni demuestra la ubicación de páginas de memoria.

El entorno registró GCC 13.3.0, Open MPI 4.1.6 y Python 3.13.11 en la preparación. Las versiones posteriores de bibliotecas se identifican en cada paquete. Los resultados WSL con Ryzen 5 2600 y GCC 15.2.0 no se utilizaron como denominadores del clúster. Se conservaron fuentes con hashes y se compiló en destino.

El paquete preformal permite verificar cinco fuentes, logs de compilación vacíos y 53 CSV originales con salidas de error vacías. Las comprobaciones pequeñas de N=128 mantienen residuo exacto consistente con las pruebas anteriores. Los paquetes conservan las instantáneas de los fuentes utilizados en cada campaña. Los archivos actuales incorporan una refactorización didáctica posterior y no coinciden byte a byte con esas versiones. Los resultados corresponden a los fuentes archivados; su conservación permite identificar el código medido. La evolución del código se describe en docs/inventario_tecnico.md.

11.1. Pilotos para elegir una campaña acotada

Los ensayos preformales incluyeron tamaños pequeños y N=512,1024,2048,4096. Sirvieron para localizar el régimen de cruce, probar que los tamaños mayores eran ejecutables y observar diferencias entre P=3,4,6. Los escaneos de una observación no se presentan como estimaciones estadísticas equivalentes a una campaña formal.

El control de N=4096/P=6 tuvo tres observaciones por método: alrededor de 10,327 s de aplicación secuencial y 4,123 s MPI. Ese control justificó continuar, pero no reemplazó la campaña posterior. El paquete preformal conserva los CSV y los fuentes, pero no un registro completo de afinidad y cgroup. Las tablas iniciales tenían valores redondeados; los cálculos actuales utilizan los CSV originales de data/raw/cluster/evidencia_preformal_20260928.tar.gz. docs/recursos_actuales.md identifica los registros disponibles en cada paquete.

12. Cruce formal del tiempo de aplicación

Se ensayaron N=128,160,192,224,256 y seis métodos: secuencial y MPI P=1,2,3,4,6. Se conservaron 30 preparaciones y 210 observaciones útiles, siete por configuración. El protocolo anunciado fijaba el secuencial, solicitaba binding para MPI y rotaba el orden. El paquete conserva las rondas y evidencia de entorno, pero no el script original de lanzamiento completo; esa ausencia limita la auditoría de determinadas condiciones.

El criterio descriptivo fijado fue S>1,10 y Q3_MPI<Q1_SEQ. Exige una ganancia de más del 10 % y separación de los intervalos intercuartílicos observados. No es un test formal de significancia, no produce p-values y no convierte siete mediciones de una sesión en garantía universal.

P	Primer N ensayado que cumple
1	ninguno
2	256
3	192
4	224
6	256

En N=128 y N=160 ningún P cumplió ambas condiciones. N=192 favoreció P=3; N=224 favoreció P=3 y P=4; en N=256 las cumplieron P=2,3,4,6. La expresión correcta es primer N ensayado: no se midieron todos los tamaños intermedios.

Método	Mediana ms	Q1 ms	Q3 ms	S	E %
seq	2.206	2.177	2.219	1.000	-
p1	2.538	2.480	2.556	0.869	86.9
p2	1.360	1.352	1.374	1.623	81.1
p3	1.121	1.112	1.147	1.968	65.6
p4	1.160	1.154	1.194	1.902	47.6
p6	1.505	1.493	1.508	1.466	24.4

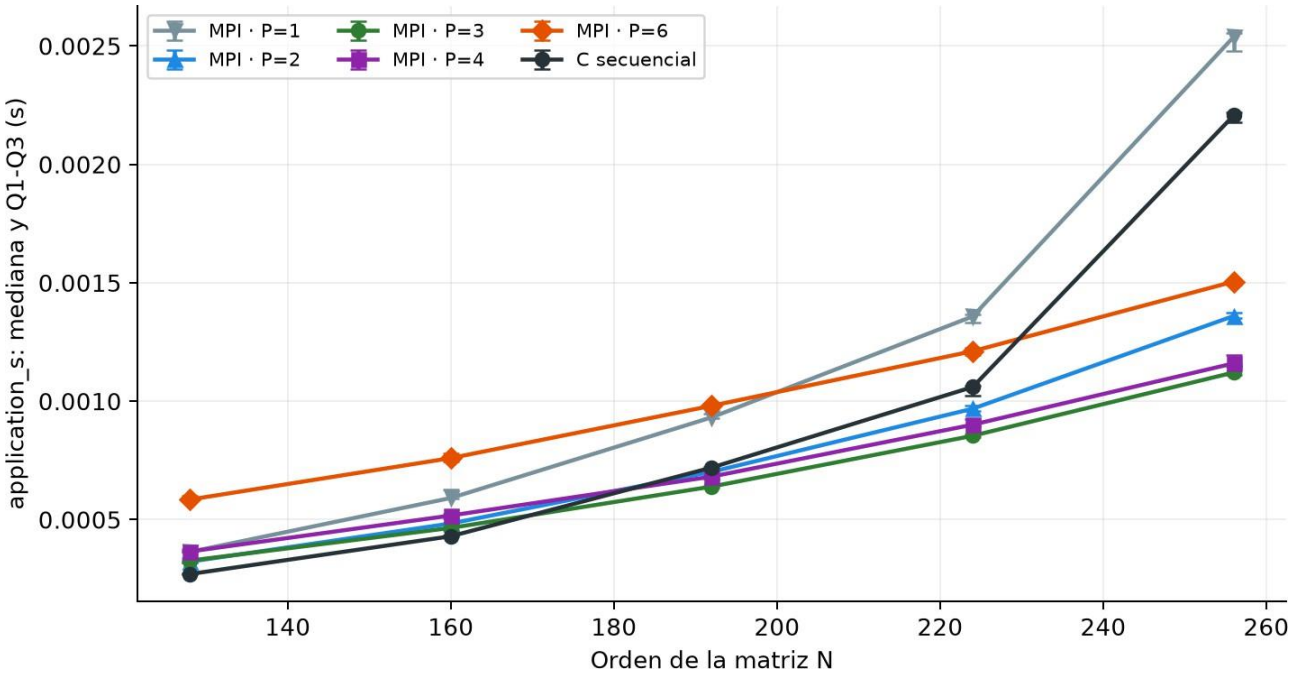


Figura 8. Medianas y Q1-Q3 de aplicación en el cruce formal; siete observaciones por punto, preparaciones excluidas.

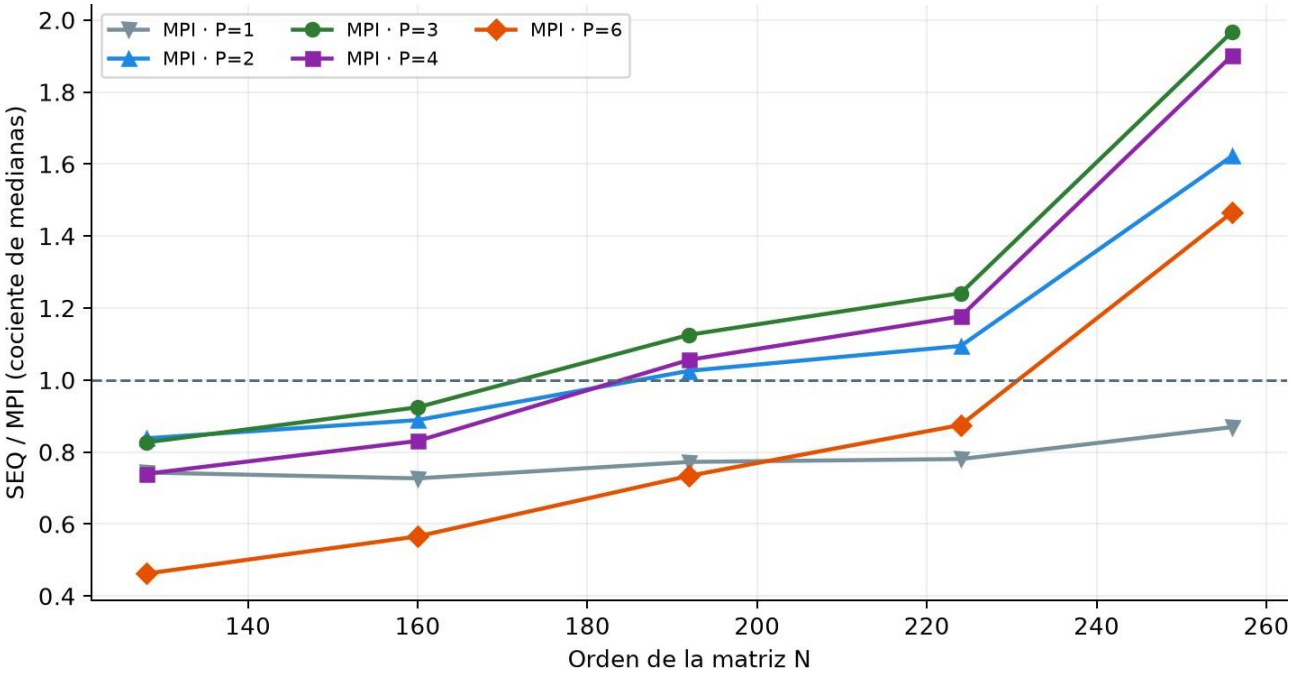


Figura 9. Cocientes de medianas del cruce, cada uno con el secuencial de su mismo N. No son intervalos de confianza.

P=3 tuvo la menor mediana en N=256, mientras que P=2 alcanzó la mejor eficiencia entre las configuraciones que aceleraron. P=6 mejoró al secuencial, pero con una eficiencia menor. No se deduce un P óptimo universal. El MPI P=1 es una ruta con generación distribuida y reunión, no una copia del ejecutable secuencial.

Las fases permiten interpretar el cruce: en N=160/P=3, generación y factor ya reducían su duración respecto de la referencia, pero la reunión todavía compensaba ese ahorro. En N=192/P=3, el balance del flujo fue favorable. Esto es consistente con amortizar los costos adicionales al aumentar trabajo, sin pretender aislar un único mecanismo causal.

Los nombres de archivo acumularon prefijos rep. La explicación registrada fue una asignación rep="$3" en una función Bash sin variable local que afectó al bucle exterior. Los CSV y sus rondas pudieron identificarse inequívocamente; cada configuración conservó siete observaciones. Se preservan los nombres y se añade su correspondencia normalizada. La corrección propuesta consiste en usar una variable local distinta. Los nombres posteriores están normalizados, pero el script corregido no se conservó y esa modificación no puede verificarse directamente.

13. Escalabilidad y beneficio del comando completo

La campaña siguiente amplió N a 512,1024,2048,4096 y mantuvo P=1,2,3,4,6 y el secuencial. Cada configuración tuvo una preparación y siete observaciones útiles: 168 mediciones y 24 preparaciones. Cada lanzamiento registra simultáneamente el reloj de aplicación y el externo. Los 192 casos tienen valid=1, código de salida cero y salida de error vacía.

N	App P6 (s)	S app	E app %	Ext P6 (s)	S externo
512	0.005	3.228	53.8	0.516	0.059
1024	0.028	3.692	61.5	0.569	0.262
2048	0.197	5.828	97.1	1.046	1.429
4096	4.134	2.514	41.9	6.601	1.873

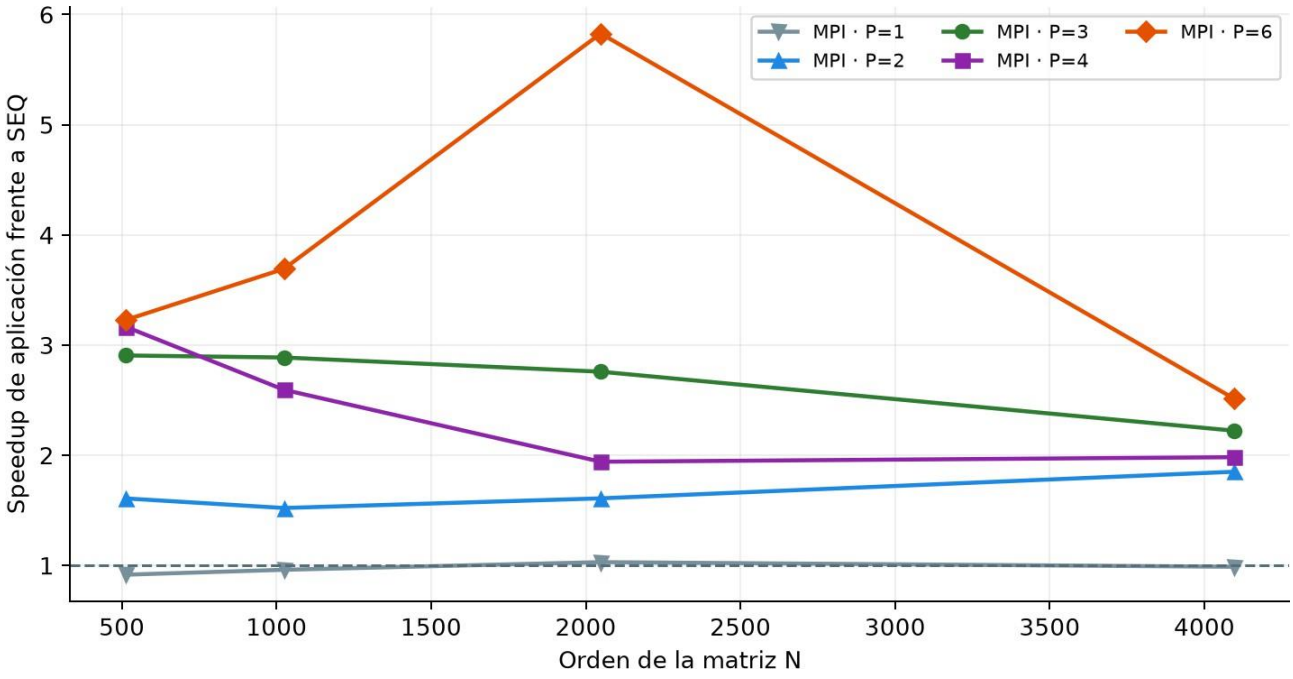


Figura 10. Speedup de aplicación para cada P en la campaña de escalabilidad. Las referencias se recalculan dentro de esta campaña; no se empalman con las del cruce.

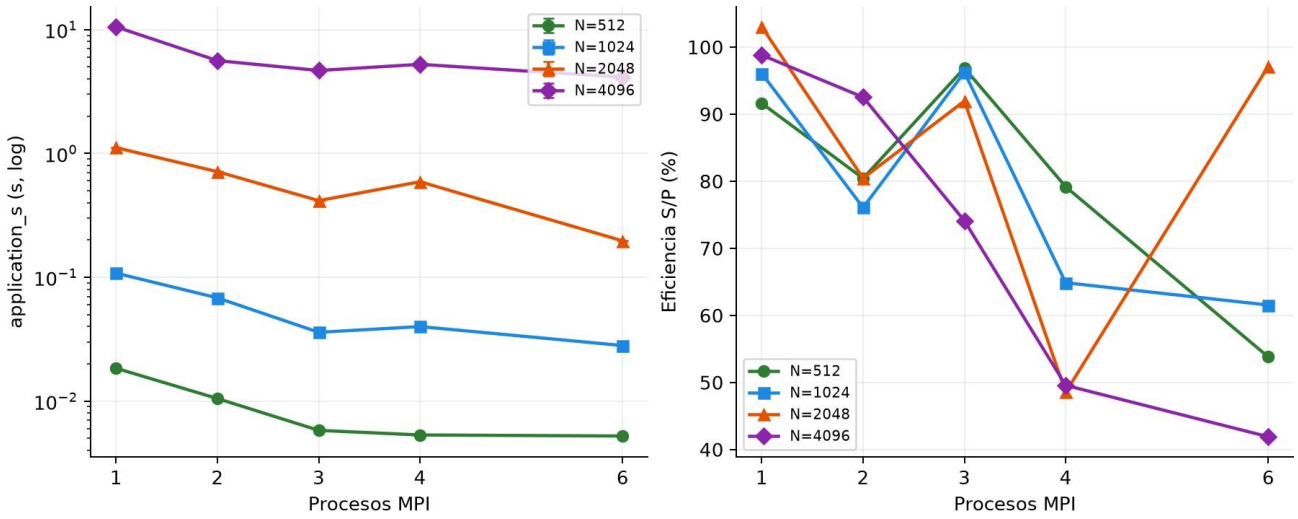


Figura 11. Tiempo de aplicación con Q1-Q3 y eficiencia S/P. Mantener N fijo y variar P permite estudiar escalabilidad fuerte dentro de la asignación.

El speedup de aplicación para P=6 alcanzó aproximadamente 3,23; 3,69; 5,83 y 2,51 para los cuatro tamaños. El valor de N=2048 no demuestra una causa específica como caché, y su descenso en N=4096 impide describir una mejora monótona con N. P=4 volvió a presentar casos peores que P=3. Comunicación, sincronización, ubicación, memoria y condiciones del entorno son factores plausibles, pero no fueron aislados causalmente.

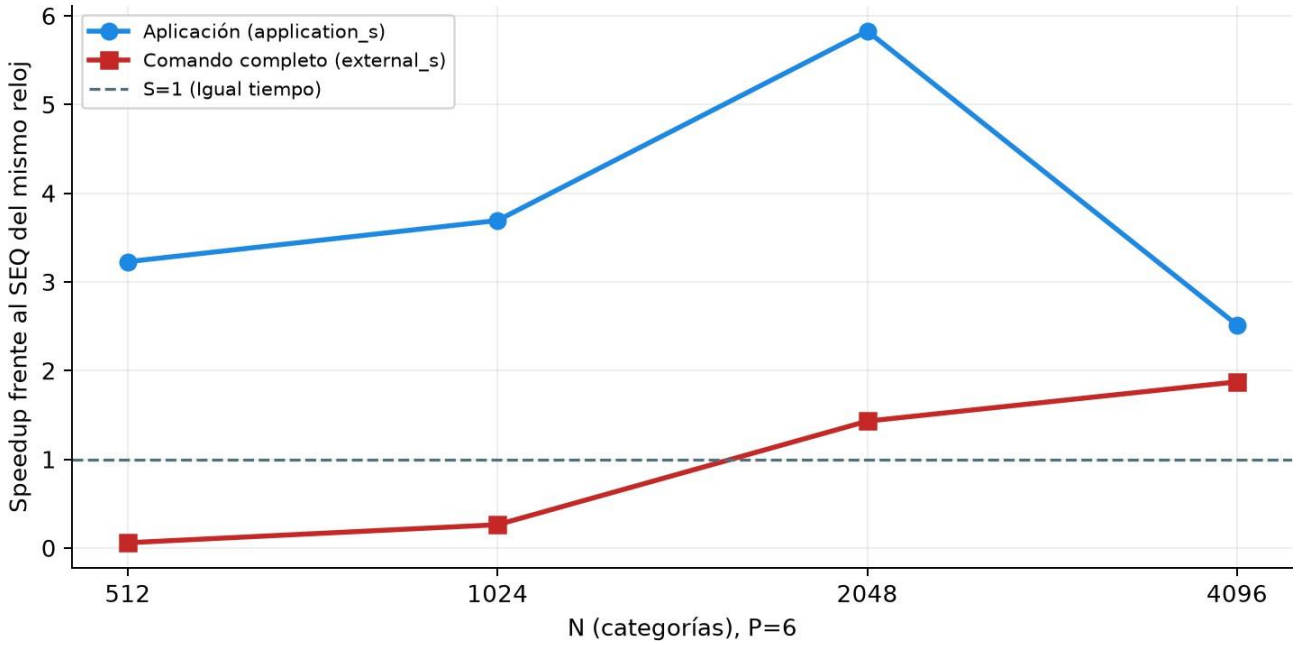


Figura 12. P=6 frente al secuencial propio: el mismo lanzamiento conduce a distintas conclusiones según la frontera temporal. La línea S=1 marca igualdad de medianas.

Para N=512 y N=1024 el comando completo MPI seguía perdiendo pese a acelerar la aplicación. En N=2048 y N=4096 la ventaja sobrevivió al reloj externo, con speedups aproximados de 1,43 y 1,87. Esos cocientes no deben presentarse como aceleraciones del kernel ni el primero como un umbral exacto de extremo a extremo: la grilla tiene cuatro tamaños separados.

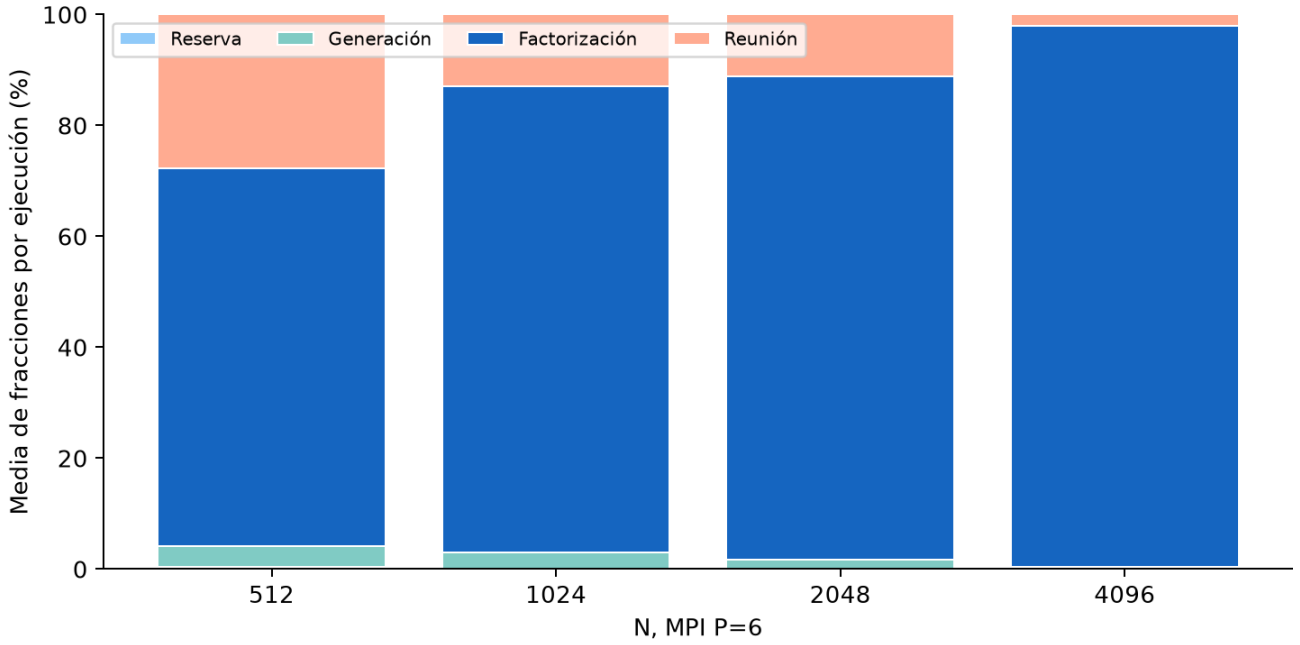


Figura 13. Composición de aplicación para P=6. Se calcula la fracción de cada fase por ejecución y luego su media, de modo que la barra suma 100 %. No se suman medianas para construir un total ficticio.

La campaña permitió distinguir la aceleración de la aplicación del beneficio del comando completo con la cuota de seis CPUs equivalentes disponible en esa sesión. Una asignación mayor requiere una campaña independiente, como la extensión del 03/10.

14. La referencia optimizada en el clúster

El entorno de biblioteca quedó identificado como LAPACK/LAPACKE 3.11.0 y OpenBLAS 0.3.34 con backend pthreads y selección de CPU Zen. El ejecutable fuerza un hilo y registra la comprobación. La variable OMP_NUM_THREADS=1 no demuestra que el candidato use OpenMP; el código propio no contiene regiones OpenMP y la biblioteca utilizada informa pthreads.

La campaña de referencia incluyó N=2048 y N=4096, seis observaciones por método dentro de compare_lapack.c. Un control N=512 se conserva aparte. Los métodos alternan la primera posición, con una preparación interna por método no exportada al CSV. La matriz original y un factor propio de referencia se preparan fuera del reloj; cada observación medida reserva, genera y factoriza una matriz nueva.

N	Reloj	C propio (s)	LAPACK (s)	Propio/LAPACK
2048	factor_s	1.060705	0.081292	13.048
2048	total_s	1.089732	0.110383	9.872
4096	factor_s	9.854616	0.589960	16.704
4096	total_s	10.269754	1.006013	10.208

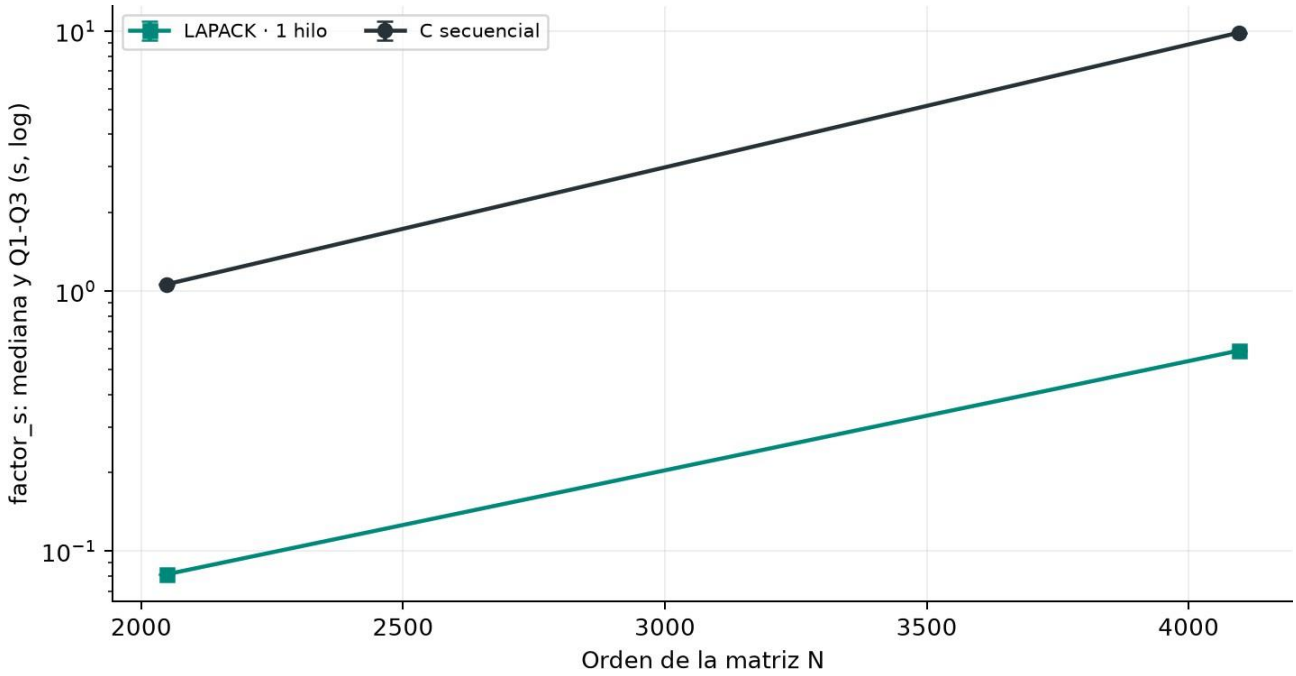


Figura 14. Medianas y Q1-Q3 de factor_s para C propio y LAPACK/OpenBLAS de un hilo, en su campaña común. Eje vertical logarítmico.

La ventaja de factorización de LAPACK fue de aproximadamente 13,05 veces para N=2048 y 16,70 para N=4096. En el total del banco fue 9,87 y 10,21. La relación en N=2048 fue del mismo orden que la observada localmente, pero no se compararon tiempos absolutos de dos máquinas como si fueran simultáneos.

El log de compilación registra una advertencia _Float16 con -Wpedantic en un encabezado de OpenBLAS. No es un error del cálculo propio. Las pruebas pequeñas registran PASS; los residuos y diferencias del factor cumplen el criterio. Los CSV principales no exportan valid ni código de salida. La comprobación numérica se apoya en los residuos y las diferencias del factor; estos datos no permiten verificar por sí solos el estado de salida del comando.

Los registros de entorno conservados identifican la presencia y las versiones de las dependencias utilizadas. No incluyen el historial completo de instalación mediante Conda ni permiten determinar qué directorios se restablecieron al reiniciar la instancia. docs/recursos_actuales.md enumera los registros disponibles; docs/guia_paso_a_paso_experimentos.md describe la ejecución de los bancos. Antes de repetir una campaña, deben verificarse las dependencias de la nueva sesión.

15. Comparación final de problemas grandes

La referencia LAPACK/OpenBLAS permitió cuantificar la distancia entre el kernel propio y una implementación local optimizada. A continuación se examinó si la aceleración obtenida mediante MPI reducía esa diferencia en tamaños mayores. Se ampliaron los tamaños a N=6144 y N=8192 y se compararon C propio secuencial, MPI P=6 y LAPACK/OpenBLAS de un hilo. No se repitió toda la grilla de P, ya estudiada. Se conservaron seis observaciones útiles por método y tamaño, 36 en total; las dos preparaciones MPI aparecen en CSV y las cuatro preparaciones propias/LAPACK están implementadas dentro del banco y no se exportan.

La métrica principal es factor_s. El secuencial y LAPACK se alternan en un proceso con repeticiones internas; MPI utiliza lanzamientos separados. La campaña común, el generador y los fuentes coincidentes hacen pertinente comparar la fase matemática de factorización, pero no garantizan idéntico estado de caché ni idéntico contexto de ejecución. application_s MPI y total_s del banco se conservan únicamente como contexto.

N	Método	n obs.	Q1 (s)	Mediana (s)	Q3 (s)
6144	own	6	27.260189	27.269004	27.271326
6144	p6	6	16.090931	16.298454	16.492704
6144	lapack	6	1.898735	1.899152	1.899557
8192	own	6	72.001985	72.091515	72.124989
8192	p6	6	36.921419	36.953931	36.994156
8192	lapack	6	5.912865	5.915935	5.918062

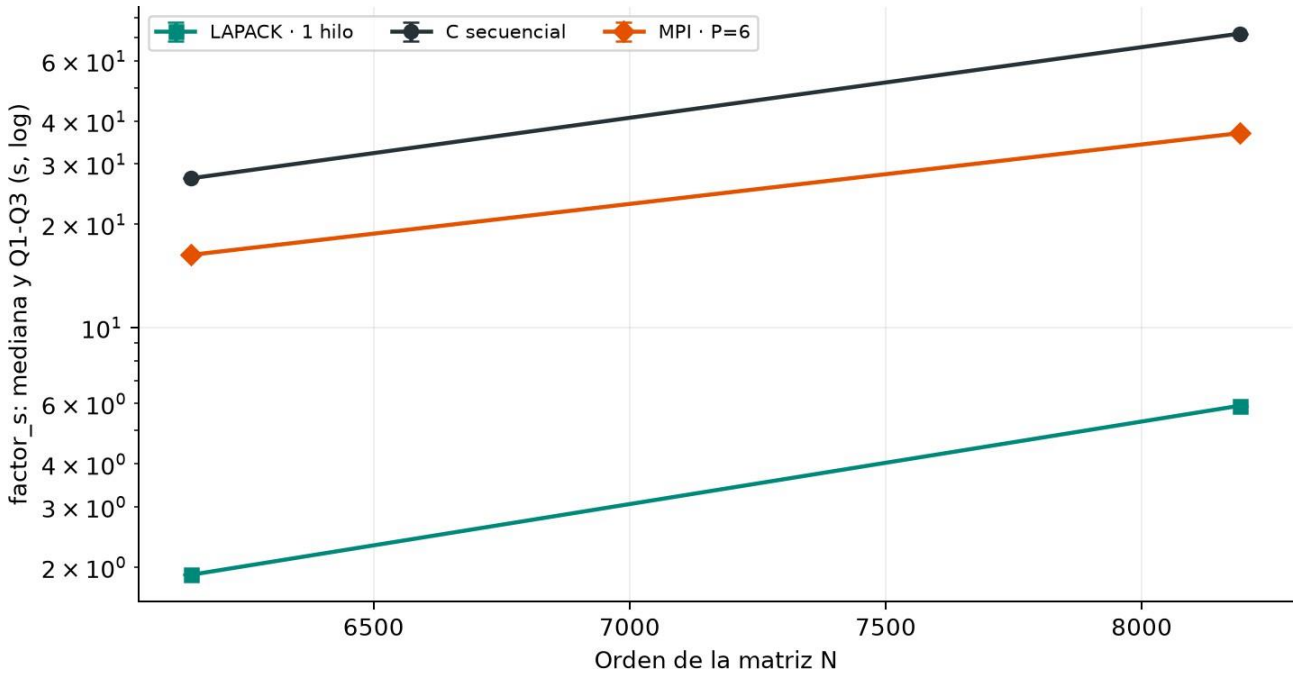


Figura 15. factor_s para los tres métodos en la campaña final, seis observaciones por grupo. Se muestran medianas y Q1-Q3; escala logarítmica.

N	C/MPI	E MPI %	C/LAPACK	MPI/LAPACK
6144	1.673	27.9	14.359	8.582
8192	1.951	32.5	12.186	6.247

MPI P=6 redujo el tiempo propio con speedups de 1,673 y 1,951, equivalentes a eficiencias respecto de esa base de aproximadamente 27,9 % y 32,5 %. Sin embargo, tardó 8,58 y 6,25 veces el tiempo de LAPACK de un hilo. MPI redujo el tiempo respecto del secuencial propio, aunque en esta campaña no alcanzó el rendimiento de LAPACK/OpenBLAS de un hilo. Este resultado compara implementaciones, no alternativas tecnológicas excluyentes.

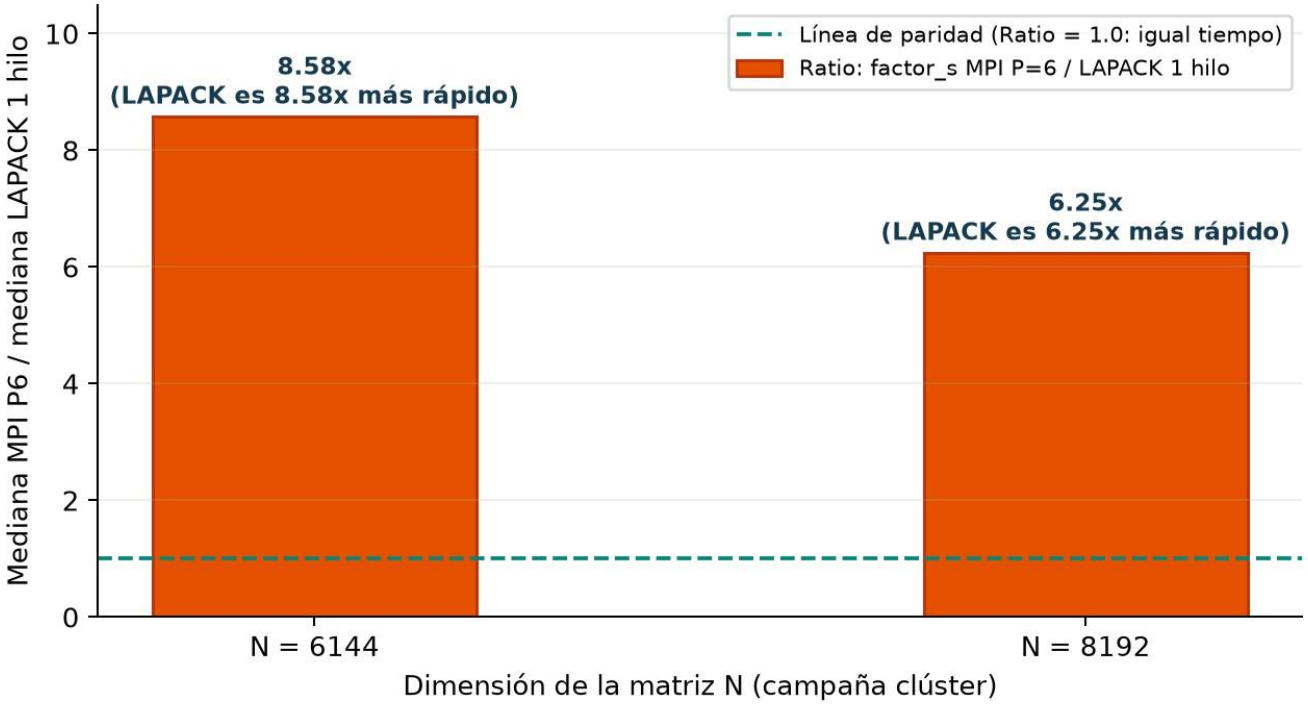


Figura 16. Cociente de medianas MPI/LAPACK en los dos tamaños finales. Un valor superior a uno significa que MPI tarda más.

La distancia relativa disminuyó entre esos dos puntos, pero la diferencia absoluta de segundos aumentó. No existe evidencia suficiente para extrapolar un cruce, predecir su tamaño ni afirmar una convergencia monótona. Hasta el mayor N ensayado, LAPACK/OpenBLAS de un hilo mantuvo menor mediana de factorización.

15.1. Observación separada sobre la generación

N	Método	Generación mediana (s)
6144	lapack	0.402331
6144	own	0.402347
6144	p6	0.132174
8192	lapack	3.978558
8192	own	3.980563
8192	p6	0.061364

La generación secuencial mostró un salto entre N=6144 y N=8192 que afecta al total del banco. El acceso simétrico por columnas y el stride asociado a una potencia de dos motivan una hipótesis de memoria/caché, pero no se midieron contadores para comprobarla. Se registra la anomalía sin atribuirle causalidad y sin mezclarla con el resultado principal de factor_s. Estudiarla sería una extensión, no una condición para cerrar esta comparación.

16. Extensión de strong scaling hasta 32 procesos

El 03/10/2026 se realizó una campaña independiente para responder cuánto mejora el mismo candidato MPI al mantener N=8192 y aumentar P a 1, 2, 4, 8, 16 y 32. Esta sesión tuvo una cuota de 48 CPUs equivalentes y 192 GiB, distinta de la asignación de seis CPUs y 24 GiB del 28/09. Sus tiempos no se agregan a las campañas anteriores ni se usan para recalcular la comparación con LAPACK.

Se ejecutó un warmup por P y siete observaciones útiles por configuración, 42 en total. El orden se rotó entre repeticiones y la última invirtió la secuencia. El binding de control situó los ranks 0 a 31 en los cores 0 a 31 del socket 0 del mismo host visible. No hubo incrementos de nr_throttled ni de throttled_usec durante la campaña formal.

factor_s sigue siendo el intervalo registrado por el rango 0 alrededor del kernel distribuido. Incluye comunicación y espera dentro de la llamada, pero no es un máximo reducido entre rangos. application_s abarca reserva, generación, factorización y reunión; la validación queda fuera. Para los tamaños grandes exact_residual=-1 significa que no se calculó la reconstrucción cúbica completa. Las 42 observaciones formales tienen valid=1 y un residuo proyectado de 3,2403e-15.

P	n	Q1 (s)	Mediana (s)	Q3 (s)	Speedup	Eficiencia %
1	7	64.085799	64.109724	64.181409	1.000	100.00
2	7	40.880406	40.909027	40.924548	1.567	78.36
4	7	36.974880	36.998314	37.028154	1.733	43.32
8	7	39.141549	39.229483	39.326518	1.634	20.43
16	7	23.784437	23.787586	23.792451	2.695	16.84
32	7	13.537468	13.543594	13.548740	4.734	14.79

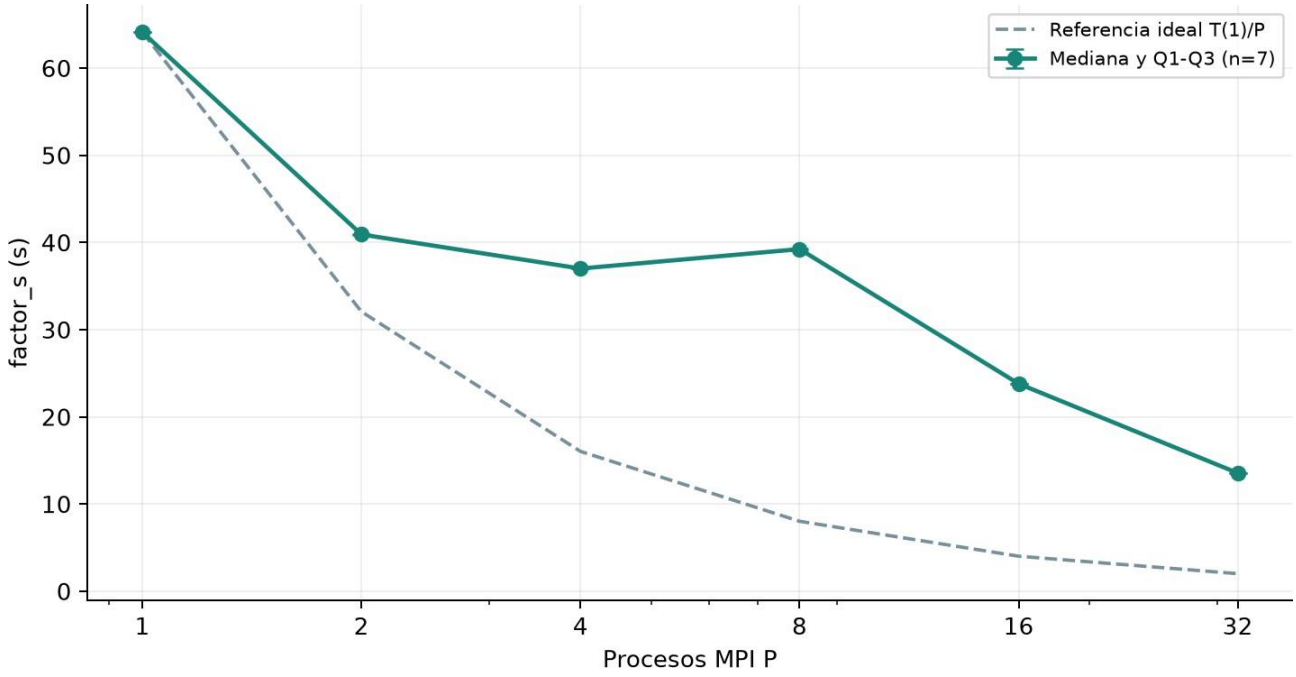


Figura 17. Mediana de factor_s y Q1-Q3 para siete observaciones por P. La línea discontinua es una referencia ideal, no una predicción.

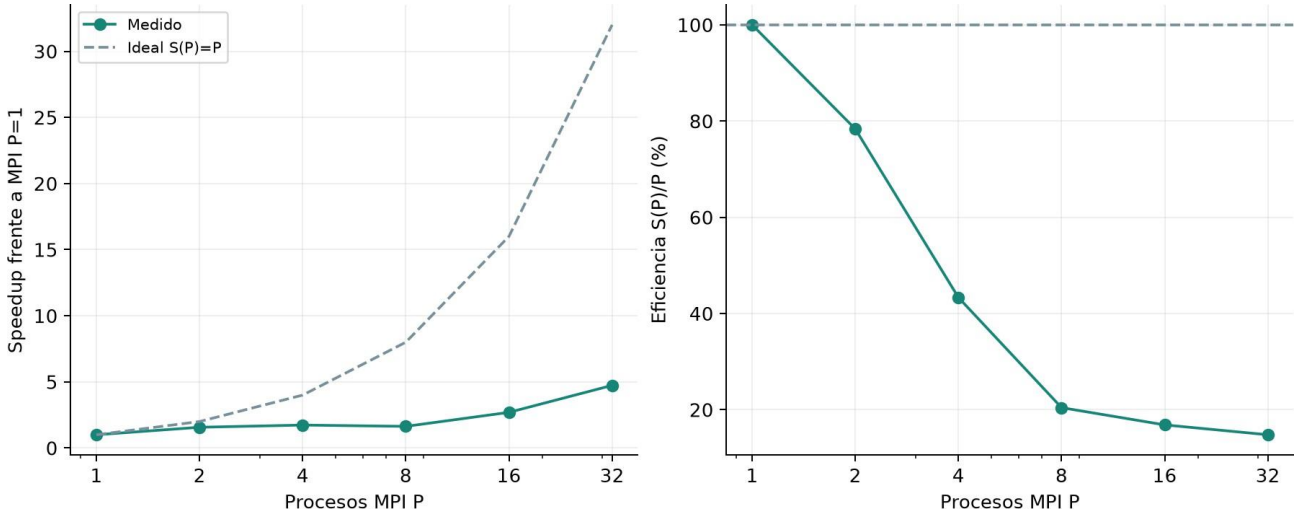


Figura 18. Speedup respecto de MPI P=1 de la misma sesión y eficiencia S/P. P=32 alcanza 4,734 veces de speedup y 14,79% de eficiencia.

La curva no es monótona. La mediana pasa de 36,998314 s con P=4 a 39,229483 s con P=8, una degradación de 6,03%. Los intervalos Q1-Q3 y los rangos mínimo-máximo no se solapan en esta muestra; además, las siete comparaciones por repetición tienen el mismo signo. Es evidencia descriptiva de esta sesión, no una identificación causal.

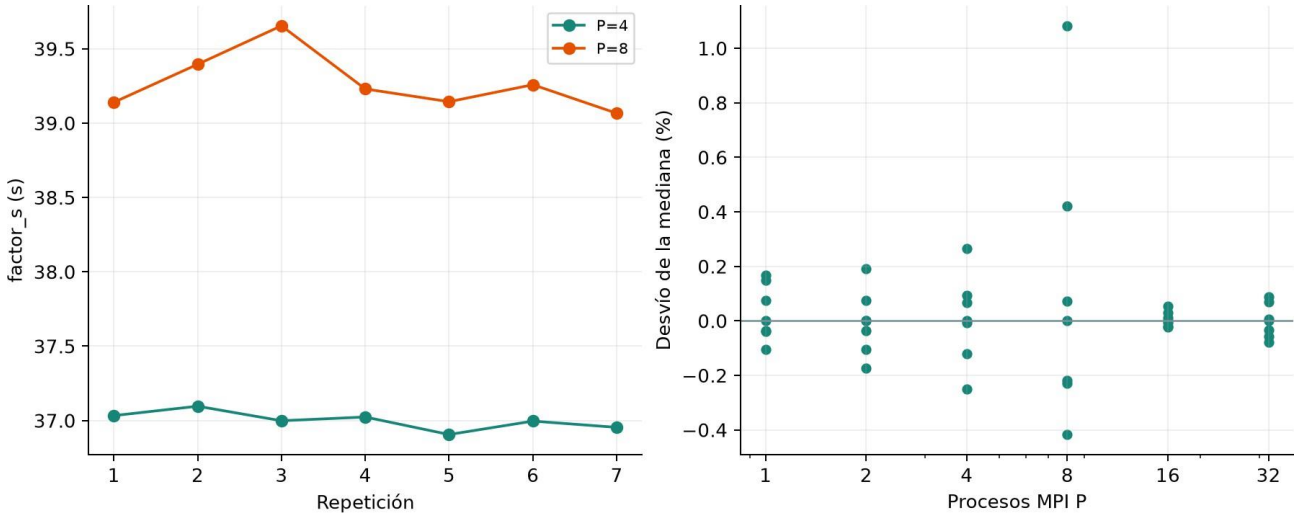


Figura 19. Repeticiones de P=4 y P=8 y dispersión relativa para todas las configuraciones. No se instrumentaron colectivas, caché ni desbalance por rango para atribuir la anomalía.

P	application_s mediana	Q1 (s)	Q3 (s)	Speedup
1	74.392908	74.349503	74.478477	1.000
2	44.068331	44.035932	44.084532	1.688
4	37.636445	37.602782	37.651906	1.977
8	39.863072	39.769258	39.956301	1.866
16	24.408448	24.404890	24.415617	3.048
32	14.185659	14.182349	14.192528	5.244

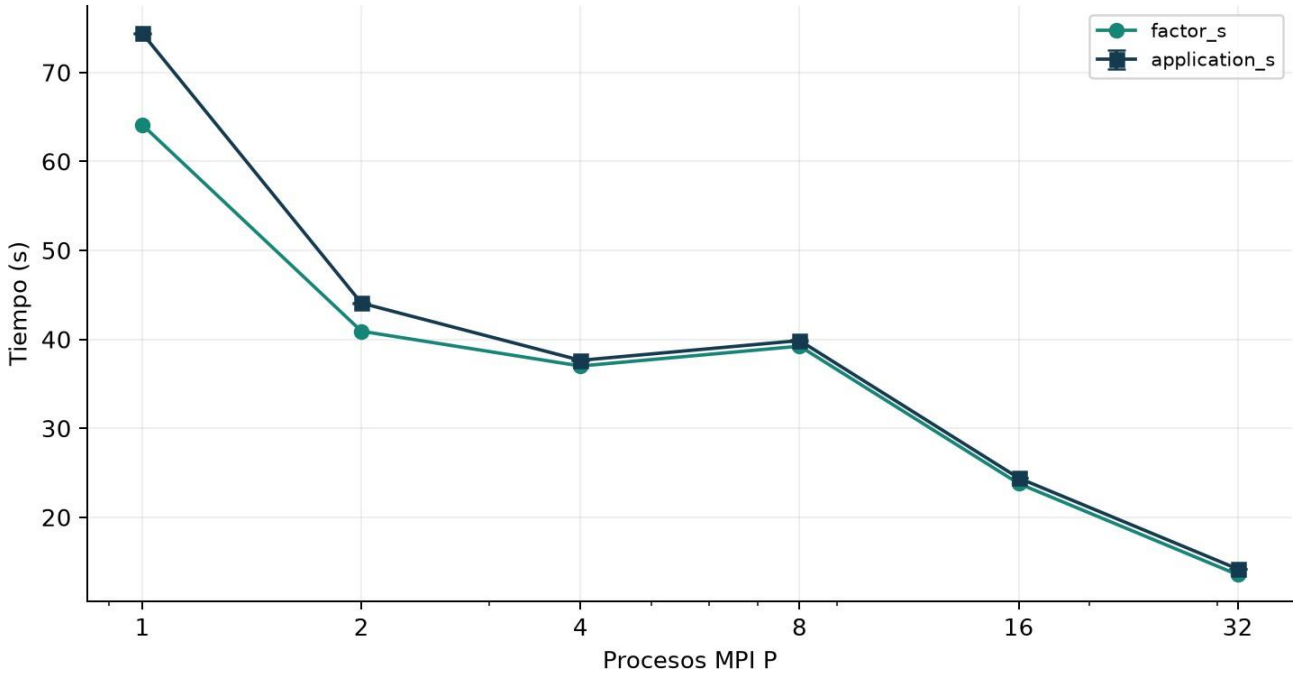


Figura 20. application_s y factor_s de la campaña formal. La mejora de aplicación no debe confundirse con el tiempo externo del comando.

El piloto N=16384 conserva una sola observación por P. P=32 vuelve a ser el menor tiempo observado y P=8 vuelve a superar a P=4, pero no se asignan barras de dispersión ni se presenta como una campaña formal.

N	P	factor_s (s)	Speedup	Eficiencia %
8192	1	63.998495	1.000	100.00
8192	2	40.887135	1.565	78.26
8192	4	36.984363	1.730	43.26
8192	8	39.218771	1.632	20.40
8192	16	23.794171	2.690	16.81
8192	32	13.552484	4.722	14.76
16384	1	460.668257	1.000	100.00
16384	2	307.788079	1.497	74.84
16384	4	285.529275	1.613	40.33
16384	8	288.170208	1.599	19.98
16384	16	178.517714	2.581	16.13
16384	32	122.774187	3.752	11.73

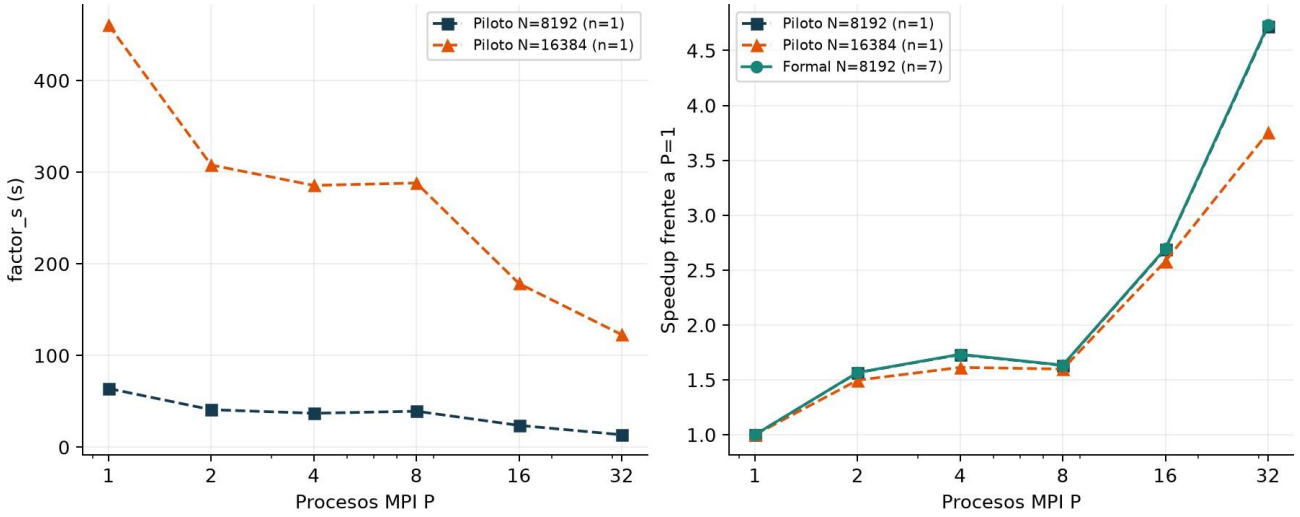


Figura 21. Pilotos N=8192 y N=16384 con una observación por configuración, diferenciados de la campaña formal.

La campaña no demuestra comportamiento multinodo, costo de red, un óptimo global en P=32 ni una causa para la degradación de P=8. También pertenece a una nueva versión experimental: los hashes archivados no coinciden con la instantánea histórica de septiembre, incluso al normalizar CRLF a LF. Por eso se conserva como extensión separada con fuentes, binario, comandos, hashes y resultados propios.

17. Corrección numérica e integridad de la evidencia

El residuo completo utilizado en problemas pequeños es $||A-LL^T||F/||A||F$. Se reconstruye el producto con acumulación long double; que sea completo no significa aritmética exacta. Su costo de orden N^3 motivó limitarlo a pruebas pequeñas. En el candidato se exporta para N<=128 y se escribe -1 cuando no se calcula; -1 no es un residuo negativo medido.

Para tamaños grandes se utiliza el máximo de $||Ax-L(L^Tx)||2/||Ax||2$ sobre ocho vectores de signos con semilla 91827. Es una comprobación reproducible de costo $O(8N^2)$, no el residuo de Frobenius ni un certificado para todas las direcciones. El candidato también comprueba valores finitos y diagonal positiva. Los bancos de comparación contrastan además el factor con una referencia propia mediante el máximo de |L-Lref|/max(1,|Lref|) por elemento del triángulo inferior.

$$r = \frac{\|A - LL^T\|_F}{\|A\|_F}$$

El umbral operativo principal es 1e-12. Algunos primeros bancos exigían 1e-13 para la diferencia de factores, sin cambiar el umbral del residuo. Las matrices ensayadas pasan los controles; esto no demuestra estabilidad universal frente a mal condicionamiento. Se registran los máximos, incluyendo preparaciones exportadas y controles cuando forman parte del conjunto auditado:

Campaña	Filas auditadas	Residuo completo máx.	Proyectado máx.	Diferencia máx.
cierre_local_20260924	223	3.790e-16	1.611e-15	-
cierre_local_20260927	222	3.790e-16	1.611e-15	-
cluster_crossing	240	3.790e-16	5.764e-16	-

Campaña	Filas auditadas	Residuo completo máx.	Proyectado máx.	Diferencia máx.
cluster_lapack	36	-	2.275e-15	6.595e-15
cluster_large	38	-	3.240e-15	8.866e-15
cluster_scalability	192	-	2.275e-15	-
cluster_strong_8192	48	-	3.240e-15	-
cluster_strong_pilot_16384	6	-	4.634e-15	-
cluster_strong_pilot_8192	6	-	3.240e-15	-
cluster_strong_trace	1	3.790e-16	3.896e-16	-
left_right_mpi_20260922	320	1.213e-15	-	2.658e-15
memoria_20260921	216	1.213e-15	-	-

Las pruebas locales archivadas incluyen sanitizadores. En MPI se documentó una incompatibilidad de ASan durante MPI_Init con el componente memory/patcher y se utilizó una configuración de prueba con ese componente desactivado. La detección de fugas estaba desactivada; no se presenta como una prueba de ausencia de fugas ni se traslada ese ajuste a los benchmarks ordinarios.

La auditoría de paquetes verifica 113 entradas del manifiesto preformal; 493 del manifiesto global del cruce y nueve adicionales de procedencia; doce de procedencia en escalabilidad; cuatro hashes del entorno LAPACK; y 43 entradas del manifiesto final. La campaña de escalabilidad y la de LAPACK no incorporan manifiestos globales de los CSV. Sus hashes de recepción identifican los paquetes actuales, pero no sustituyen una firma o un manifiesto previo ausente. Los ejecutables incluidos se preservan como evidencia y no se ejecutan durante el análisis.

18. Condiciones del entorno y límites de atribución

Campaña	Delta períodos	Delta throttled	Delta throttled_usec
cruce	941	0	0
escalabilidad	5514	197	764111
final_grande	15888	1676	4376492
strong_scaling_formal_8192_20261003.tar.gz	25276	0	0
strong_scaling_pilot_16384_20261003.tar.gz	20478	0	0
strong_scaling_pilot_8192_20261003.tar.gz	3158	0	0

Los contadores de cgroup rodean campañas completas y pueden incluir actividad auxiliar. En el cruce no aumentaron los contadores de throttling. En escalabilidad y tamaños grandes sí hubo incrementos. No permiten asignar un costo a un P concreto, corregir los tiempos restándolo ni explicar por sí solos la

anomalía de P=4. El diagnóstico preformal de N=4096 conserva mediciones en evidencia_preformal_20260928.tar.gz, pero el paquete no contiene los contadores necesarios para verificar sus incrementos de throttling.

Las campañas se realizaron dentro de una asignación de un host; no se ensayó comunicación entre nodos. Los diagnósticos de binding no garantizan exclusividad ni control NUMA de páginas. La rotación anunciada de algunas campañas del clúster no puede auditarse por completo cuando falta el script; la alternancia de compare_lapack.c, en cambio, está presente en el código archivado y en los campos de orden.

Se utilizaron seis o siete repeticiones por grupo en las campañas formales principales. Medianas y cuartiles describen esa muestra, no intervalos de confianza. No se realizaron pruebas de hipótesis ni se estimó incertidumbre inferencial de los cocientes. Las repeticiones de una misma matriz caracterizan variabilidad temporal, no una población de matrices independientes.

La validación centralizada, los buffers completos y la salida reunida definen el caso estudiado y limitan su escalabilidad de memoria. El candidato carece de entrada de matrices externas, distribución 2D, almacenamiento triangular compacto y kernel BLAS-3. Esas decisiones de alcance se describen para interpretar los resultados, sin convertir cada extensión posible en un requisito pendiente.

19. Evaluación en una segunda computadora

Se agregó una campaña independiente, realizada en una computadora Intel Core i5-1035G1 (4 núcleos físicos, 8 hilos lógicos) con Ubuntu bajo WSL2 y aproximadamente 3,7 GiB de memoria disponible para WSL. Se utilizaron GCC 15.2.0, Open MPI 5.0.10, LAPACKE 3.12.1 y OpenBLAS 0.3.32. Estas mediciones no se incorporan a las distribuciones estadísticas obtenidas previamente en Ryzen o AMD EPYC.

Persistencia de la mejora del vector local

La comparación secuencial de N=1024, con 12 repeticiones por variante, volvió a favorecer right-looking con vector. La mejora frente a right-looking original fue de 3,29 veces y frente a left-looking de 1,84 veces. En la campaña original, el primer cociente había sido de 15,07. La dirección de la mejora se conserva, pero no su magnitud.

Versión secuencial	N=1024, mediana (ms)
Left-looking	230,74
Right-looking original	412,60
Right-looking con vector	125,59

Escalabilidad fuerte en la segunda computadora

Con N=4096 fijo se midieron siete observaciones útiles por número de procesos, después de un calentamiento. Al pasar de MPI P=1 a P=4, la factorización aumentó de 15,15 s a 18,00 s, un 18,8 %. El speedup interno fue menor que uno en todas las configuraciones P>1 ensayadas. Los tiempos incluyen las comunicaciones y esperas del kernel; no son tiempos de cálculo puro.

P	Mediana factor_s (s)	Q1–Q3 (s)	S frente a MPI P=1
1	15,1502	14,4356–15,2246	1,000
2	17,1442	15,6231–17,7183	0,884
3	18,3394	17,7458–19,0405	0,826
4	18,0030	17,2894–19,0941	0,842

Otros contrastes y alcance

Para N=1024/P=3, el reparto contiguo con vector local alcanzó 136,89 ms y el cíclico 155,70 ms: el orden se invirtió respecto de la campaña original que motivó la elección de reparto. Para N=2048/P=4, la generación centralizada registró 52,04 ms y la distribuida 55,96 ms; en ese caso la generación distribuida tampoco ganó en tiempo.

En N=2048, la factorización del C secuencial propio tardó 1,9282 s frente a 0,1014 s de LAPACK/OpenBLAS con un hilo, una diferencia cercana a 19 veces. En el reloj externo del candidato, el secuencial propio registró 1,9857 s y MPI P=4, 3,2448 s. Son fronteras de medición diferentes y no se combinan en un solo cociente.

El paquete conserva 634 salidas CSV previstas y una verificación de 6322 filas numéricas, sin errores reportados, con máximo residuo o diferencia de $3{,}9784 \times 10^{-15}$ (umbral 10^{-12}). La comparación de 33 fuentes no encontró diferencias con sus originales. Un script inicial terminó con código 127 por un retorno de carro después de producir tres CSV completos que pasaron la verificación.

Esta réplica amplía la validez externa del estudio: el vector auxiliar fue beneficioso en el caso repetido, pero la distribución cíclica y el escalado MPI no conservaron necesariamente su ventaja temporal. La evidencia no permite atribuir esa variación a una causa exclusiva entre CPU, memoria, comunicación, planificación y WSL2.

20. Conclusiones

La investigación confirmó que la conveniencia de MPI depende del tamaño, la implementación local y la frontera de medición. El análisis de memoria cambió primero la selección secuencial: copiar la columna activa compensó su costo y permitió adoptar right-looking con vector. El diseño MPI requirió además combinar reparto cíclico y vector local; una distribución teóricamente favorable no había sido suficiente en la primera versión.

La generación distribuida redujo el costo de preparar los datos, pero la mejora de una fase no se trasladó íntegramente al flujo. La exploración de propiedad por grupos de columnas tampoco produjo una regla estable: la ventaja inicial de b=4 desapareció en otra sesión y se mantuvo b=1. Conservar esos resultados evita seleccionar una variante por una única medición favorable.

En la campaña formal intranodo, el primer N ensayado que cumplió el criterio descriptivo de mejora de aplicación fue 192 con P=3. Los distintos P cruzaron en tamaños distintos y las curvas no fueron siempre monótonas. La campaña mayor demostró que acelerar la aplicación no implica terminar antes el comando: P=6 perdió en el reloj externo para N=512 y N=1024, pero ganó para N=2048 y N=4096.

En una sesión independiente, con una cuota de recursos diferente, la extensión de strong scaling mostró que aumentar P hasta 32 redujo sustancialmente el tiempo para N=8192, pero con eficiencia decreciente. P=32 obtuvo el menor tiempo ensayado y un speedup de 4,734 respecto de MPI P=1; no se presenta como óptimo global. La degradación entre P=4 y P=8 fue consistente dentro de la sesión, aunque su causa queda abierta por falta de instrumentación interna.

LAPACK/OpenBLAS puso en perspectiva la ganancia del paralelismo propio. Con un solo hilo, su factorización fue sustancialmente más rápida. Al ampliar a N=6144 y N=8192, MPI P=6 mejoró al secuencial propio, pero no alcanzó a LAPACK. No se concluye que LAPACK sea un algoritmo mejor que MPI: una biblioteca local optimizada y un mecanismo de distribución son componentes distintos. La evidencia muestra que repartir un kernel menos eficiente no compensa necesariamente su desventaja local.

Las campañas realizadas permiten responder las preguntas planteadas dentro del alcance estudiado. Una ampliación puede evaluar nuevas condiciones o comprobar la estabilidad de los resultados en otras sesiones, manteniendo referencias contemporáneas, relojes explícitos y la evidencia original.

A partir de las pruebas en la segunda computadora, podemos decir que el rendimiento paralelo no depende únicamente del algoritmo y del numero de procesos, sino también de las características de la arquitectura y del entorno de ejecución.

21. Continuación técnica razonable

La primera extensión sería mejorar la reutilización de datos del kernel mediante un Cholesky realmente bloqueado y operaciones BLAS-3, manteniendo los tests de corrección y comparando la fase de factorización. El antecedente de LAPACK justifica estudiar esa dirección, pero no garantiza qué fracción de su ventaja reproduciría una nueva implementación.

OpenMP constituye una continuación importante para paralelismo de memoria compartida. Una versión híbrida podría usar MPI entre nodos o dominios y menos procesos dentro de cada nodo, con hilos OpenMP para trabajo local. Habría que mantener constante el presupuesto total de recursos, controlar afinidad y evitar sobreasignar hilos si también se usa BLAS multihilo. No se implementó esa versión ni se anticipa una ganancia medida. Especificación OpenMP 5.2.

Otras extensiones son reducir almacenamiento y buffers, distribuir la validación, estudiar NUMA, medir contadores para la anomalía de generación y examinar la no monotonicidad de P=4. Las pruebas multinodo permitirían evaluar costos de red que esta campaña intranodo no identifica. ScaLAPACK, con distribuciones 2D por bloques, sería una referencia distribuida profesional para otra comparación; no participó en los experimentos presentes. Guía de ScaLAPACK.

Referencias bibliográficas y documentación técnica

Ballard, G., Demmel, J., Holtz, O. y Schwartz, O. (2009). Communication-optimal parallel and sequential Cholesky decomposition. Referencia teórica citada en las secciones de dependencias y comunicación.

Netlib. LAPACK: rutina DPOTRF y documentación de la biblioteca. https://www.netlib.org/lapack/

Netlib. ScaLAPACK Users' Guide: distribución bidimensional bloque-cíclica y almacenamiento local. https://www.netlib.org/scalapack/slug/

MPI Forum. MPI: A Message-Passing Interface Standard. https://www.mpi-forum.org/docs/

Open MPI. Documentación de MPI_Bcast y MPI_Wtime. https://docs.open-mpi.org/

OpenBLAS Project. Documentación del proyecto y configuración de hilos. https://www.openblas.net/

OpenMP Architecture Review Board. Especificaciones de OpenMP. https://www.openmp.org/specifications/